COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	**INFORMATION SECURITY**
Instructor	**MAM AMBREEN GULL**
Assignment No.	**1**

Student Details

Registration No.	Name
SP24-BSE-045	SHEIKH FARAZ ADIL

CODE:

```python
def caesar_encrypt(text, shift):
    result = ""

    for char in text:
        if char.isalpha():
            if char.isupper():
                result += chr((ord(char) - 65 + shift) % 26 + 65)
            else:
                result += chr((ord(char) - 97 + shift) % 26 + 97)
        else:
            result += char

    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)

message = input("Enter message: ")
shift = int(input("Enter shift value: "))

encrypted = caesar_encrypt(message, shift)
print("Encrypted:", encrypted)

decrypted = caesar_decrypt(encrypted, shift)
print("Decrypted:", decrypted)
```

OUTPUT:

```
Enter message: Attack At Dawn
Enter shift value: 3
Encrypted: Dwwdfn Dw Gdzq
Decrypted: Attack At Dawn
```

CODE EXPLANTION LINE BY LINE:

- Step 1: Define function caesar_encrypt(text, shift) to encrypt the message.
- Step 2: Create an empty string result = "" to store the encrypted text.
- Step 3: Use a loop for char in text: to check each character one by one.
- Step 4: Check if the character is a letter using char.isalpha().
- Step 5: If the letter is uppercase (char.isupper()), convert it to ASCII using ord(char), subtract 65, add shift, apply % 26 for wrap-around, add 65 back,
- convert to character using chr(), and add to result.
- Step 6: If the letter is lowercase, do the same process but subtract and add 97 instead of 65.
- Step 7: If the character is not a letter (space, number, symbol), add it to result without changing it.
- Step 8: After the loop ends, return the encrypted result.
- Step 9: Define caesar_decrypt(text, shift) which calls caesar_encrypt(text, -shift) to reverse the encryption.
- Step 10: Take user input for message and shift value.
- Step 11: Call the encrypt function and print the encrypted message.
- Step 12: Call the decrypt function and print the original message.

Security Analysis:

- This program implements the Caesar Cipher, which is a very basic and weak encryption technique not suitable for securing sensitive data.
- The shift key ranges only from 0–25, giving a very small key space that can easily be broken using brute force attacks.
- It is vulnerable to frequency analysis because the relative pattern of letters remains unchanged after shifting.
- The encryption process is deterministic, meaning the same message with the same shift will always generate the same encrypted output.
- The program does not provide authentication, integrity verification, or secure key management.
- It is appropriate for learning and understanding basic cryptographic concepts, but not for real-world security use.